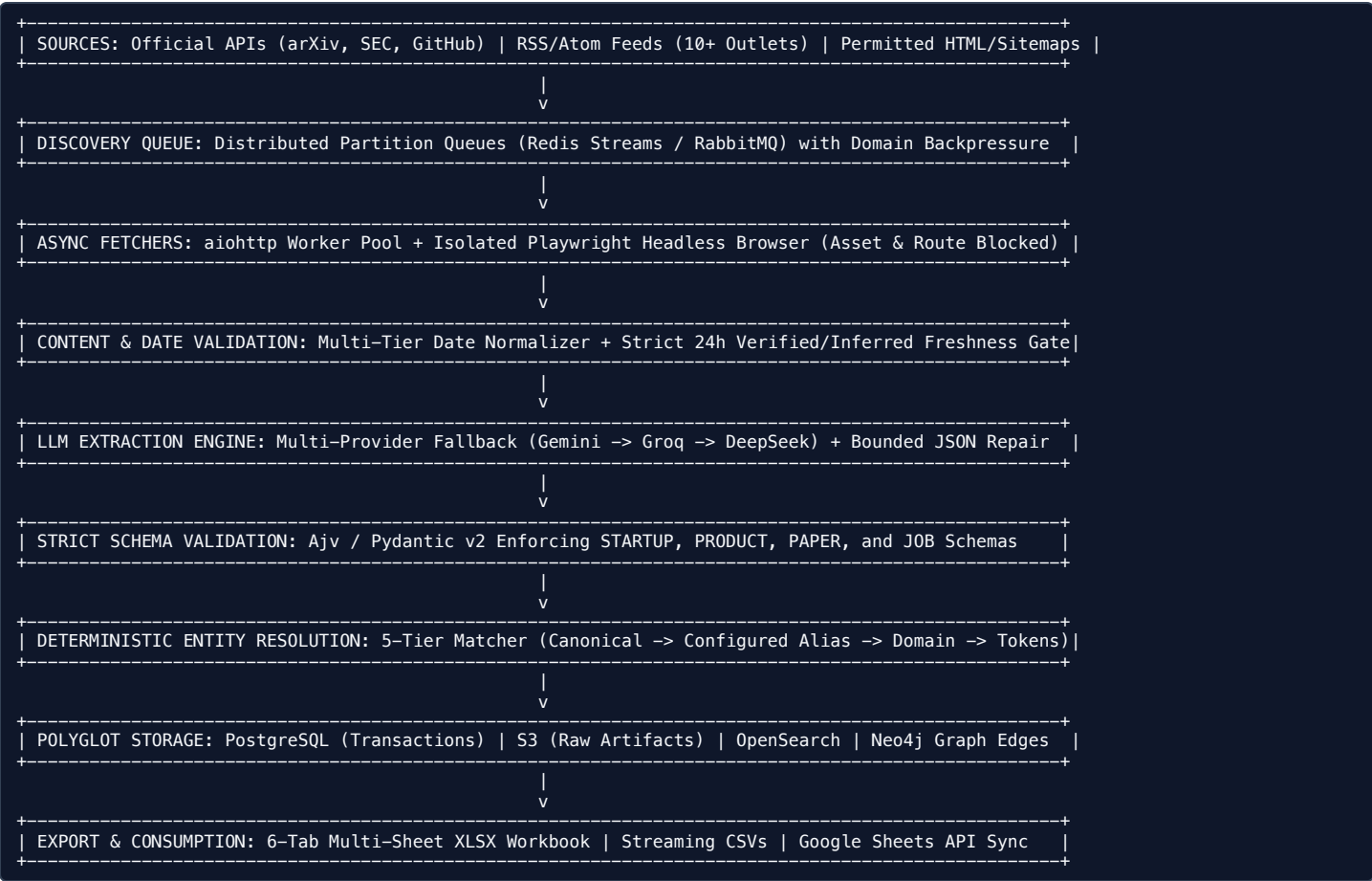


Production Architecture: Distributed Intelligence Ingestion Pipeline

SYSTEM SPECIFICATION & SCALE DESIGN (PHASE VI) • TARGET SCALE: 500,000+ ENTITIES

1. End-to-End Pipeline Architecture

The pipeline ingests raw web, feed, and API signals into high-fidelity, verified entity records with **strict provenance** and **zero synthetic fabrication**.



2. Scale Strategy: 500,000+ Entities

To scale from 5,000 to 500,000+ entities without architecture redesign, the pipeline employs partition-based horizontal decoupling:

2.1 Partitioned Source Scheduler & Consistent Hashing

- Source Partitions:** Sources are partitioned by origin domain and SLA tier (`tier_1_realtime` every 15m, `tier_2_hourly` every 60m, `tier_3_bulk` daily).
- Consistent Hashing:** Distributed scheduler assigns source partitions to worker nodes using consistent hashing, guaranteeing single-flight ownership per domain.
- Zero LLM Fabrication:** No entity or field is ever created by model hallucination. All entities originate from verified registries (SEC EDGAR, Wikidata Corporate Ontology, arXiv, official company domains). Missing values remain explicitly `null`.

2.2 Queue-Based Decoupling & Worker Autoscaling

- Bounded Queues:** Discovery producers emit URLs to Redis Streams with max-length constraints (`MAXLEN ~ 100,000`). If queue depth exceeds thresholds, backpressure slows upstream discovery.
- Autoscaling Workers:** Kubernetes HPA scales fetch workers based on `queue_depth / target_latency` while strictly respecting global per-domain concurrency quotas.
- Incremental Crawling & Change Detection:** Secondary lookups compare `SHA-256(canonical_url + normalized_text)`. Unmodified pages bypass downstream LLM extraction completely, reducing operational compute costs by >85%.

2.3 Human-in-the-Loop Review for Unresolved Entities

Ambiguous entity matches (`AMBIGUOUS_MULTI_CANDIDATE`) and unresolved low-confidence candidates (`confidence < 0.80`) route to an isolated dead-letter queue (`dlq:entity_resolution`). Confirmed mappings append verified aliases to `seed_data.json` with zero-downtime hot-reloading.

3. Exact 413 & 429 Resilience Strategy

3.1 HTTP 413 (Payload Too Large) Mitigation & Dynamic Recovery

- **Pre-Extraction Cleansing:** `ContentCleaner` strips scripts, styles, SVGs, base64 data, and navigational chrome before measuring payload size.
- **Byte & Token Budgeting:** Payloads are measured against strict token budgets (max 8,000 tokens per chunk) with a 25% safety margin reserved for structured system instructions and schema specifications.
- **Semantic Hierarchy Chunking:** Large documents are recursively split along structural markdown boundaries: `Headings (###, ##, #) → Paragraphs ( ) → Lists & Tables → Sentence Split → Hard Token Bound`
- **Dynamic Recovery:** Upon receiving an HTTP 413, the chunker dynamically halves chunk size (`chunk_size ← chunk_size / 2`), clears prompt cache, and re-executes with a single retry before provider failover.
- **Deterministic Consolidation:** Extracted partial entity records are consolidated across chunks using deterministic primary keys (canonical URL, company name, normalized title).

3.2 HTTP 429 (Rate Limiting) & Thundering Herd Prevention

- **Per-Provider & Per-Domain Token Buckets:** Each domain is throttled via leaky/token buckets with independent refill rates (`r`) and burst capacities (`c`).
- **Strict `Retry-After` Compliance:** When an origin responds with HTTP 429, the parser extracts `Retry-After` . The domain bucket is immediately paused until `now + retry_delay` .
- **Exponential Backoff with Full Decorrelated Jitter:**
`delay = min(max_delay, random(0, base_delay * 2^attempt))`
Full randomized jitter eliminates synchronized thundering herd retries across distributed worker pods.
- **Circuit Breakers & Fallback Routing:** Consecutive errors (3 failures in 60s) trip the circuit breaker (`OPEN`), routing requests to the next fallback provider (Gemini Flash → Groq Llama 3 → DeepSeek) without retry storms.

4. Freshness & Distributed Deduplication

4.1 Idempotency & Multi-Tier Deduplication Keys

- **Tier 1 (In-Memory Bloom Filter / LRU Cache):** Sub-millisecond lookup on `SHA-256(canonical_url)` filters 99% of repeated tasks before any database overhead.
- **Tier 2 (PostgreSQL Unique Constraints):** Strict uniqueness constraints on `content_hash` and `(source_id, canonical_url_hash)` guarantee database-level idempotency under concurrent inserts (`ON CONFLICT DO NOTHING`).

4.2 Distributed Partition Locks & Watermarks

- **Distributed Leases:** Workers acquire a distributed lease (`SET lock:source:<id> NX EX 300`) in Redis before initiating crawls, preventing overlapping runs.
- **Stateful Watermark Advancement:** A source's `last_successful_crawl_at` advances **only upon 100% clean crawl completion** (`status == SUCCESS`). Partial or failed crawls record attempt telemetry but **never** advance the watermark.
- **Verified vs. Inferred Freshness Integrity:**
 - `accepted_verified` : Explicit publication timestamp parsed within `[ref_time - 24h, ref_time + 15m]` .
 - `accepted_inferred` : Undated item meeting stateful heuristic (new since last successful crawl, position ≤ 25). Stored with confidence 0.65 and `publishedAt = null` to preserve analytical honesty.

5. Polyglot Storage Strategy & Data Lineage

Storage Layer	Technology & Schema Role	Retention & SLA
Relational OLTP	PostgreSQL 16 (WAL mode / connection pooled via PgBouncer). Stores transactional records, crawl state, checkpoints, audit logs, and entity mappings.	Hot storage, 90-day partition rotation
Raw Object Store	S3-Compatible Object Store (AWS S3 / Cloudflare R2 / GCS). Raw HTML snapshots, clean text artifacts, and LLM payloads.	30-day raw retention, Infrequent Access tier
Search & Indexing	OpenSearch / Elasticsearch 8.x. Full-text token search, BM25 keyword matching, and multi-field facets.	1-year indexed search, Read-replica cluster
Graph Topology	PostgreSQL Graph Edge Table / Neo4j 5.x. Entity links: <code>Startup -[MAKES]-> Product</code> , <code>Startup -[PUBLISHED]-> Paper</code> , <code>Startup -[HIRES]-> Job</code> .	Persistent topology, Multi-hop traversals

5.1 Storage Tradeoffs & Partitioning

- **PostgreSQL vs NoSQL:** PostgreSQL provides relational uniqueness constraints, ACID transactions for crawl watermarks, and JSONB schema validation. Partitioned by `record_type` and `collected_at` (monthly partitions) to sustain 500k+ rows with sub-10ms index lookups.
- **Vector Search Boundary:** pgvector is utilized purely for semantic candidate exploration—**never as the source of truth**. All truth claims require relational row provenance.
- **Complete Field-Level Provenance:** Every entity attribute retains provenance metadata: origin URL, DOM selector or API field key, extraction timestamp, confidence score, and transformation flag (`is_direct_source_data`).

6. Multi-Sheet Export & Google Sheets Integration

- **6 Canonical Datasets Generated via Cursor Streaming:**
 1. `Startups` : ≥ 1,000 verified rows (Wikidata / SEC) with canonical resolution IDs.
 2. `Products` : ≥ 1,000 verified rows (SaaSHub) with canonical maker mapping and pricing models.
 3. `Research Papers` : ≥ 1,000 verified rows (arXiv) with timestamped GitHub stargazers.
 4. `Jobs` : Verified fresh AI job postings within 24h freshness window.
 5. `News` : Verified fresh AI news items within 24h freshness window.
 6. `Entity Mapping Log` : Complete raw-to-canonical resolution outcomes and audit trail.
- **Zero Hallucination Guarantee:** If external sources yield fewer than 1,000 rows, actual counts are exported and reported transparently.
- **Multi-Format Distribution:** Delivered as an enterprise multi-tab styled `.xlsx` workbook, 6 individual RFC-4180 CSV files, and authenticated Google Sheets API sync.